



Instituto Tecnológico y de Estudios Superiores de Monterrey

Flashpoint: Fire Rescue — Simulación Multiagente

Modelación de sistemas multiagentes con gráficas computacionales (Gpo 102)

Profesores:

Pedro Oscar Pérez Murueta

Denisse L. Maldonado Flores

Alejandro Fernández Vilchis

Presenta:

Víctor Adrián García Galván – A01713062

Carlos Arturo Gómez Ayala – A01711027

Luis Fernando Martínez Barragán - A01613426

10 de septiembre de 2026

Ficha del proyecto

Nombre	Flashpoint: Fire Rescue — Simulación Multiagente
Género	Simulación multiagente de búsqueda y rescate.
Tecnologías principales	Unity 6, Python, Mesa 3.5.1 y comunicación HTTP mediante JSON
Público objetivo	Estudiantes y usuarios interesados en sistemas multiagente y simulaciones de búsqueda y rescate.

Tabla 1. Ficha del proyecto de Flashpoint: Fire Rescue

Pitch

Flashpoint: Fire Rescue es una simulación multiagente en la que seis bomberos autónomos deben localizar y rescatar a las víctimas atrapadas dentro de un edificio en llamas antes de que el fuego provoque demasiadas pérdidas o el edificio colapse. El proyecto permite comparar una estrategia de comportamiento aleatorio contra una estrategia mejorada basada en planeación de rutas, priorización de objetivos y coordinación entre agentes.

El proyecto parte del reto de desarrollar un modelo multiagente con visualización 3D para rescatar víctimas antes de que el fuego se propague o la estructura colapse.

Concepto y pilar de diseño

El proyecto toma como base las reglas Family Game Setup de Flash Point: Fire Rescue. Todos los bomberos pertenecen al mismo equipo y el resultado de la simulación depende del desempeño colectivo de los seis agentes. En lugar de controlar manualmente a cada bombero, el usuario observa cómo los agentes toman decisiones autónomas dentro de un escenario dinámico.

Pilar de diseño

Cada decisión debe equilibrar el rescate inmediato con el riesgo de que el incendio empeore.

Este principio guía la estrategia mejorada. En algunos casos, dirigirse inmediatamente hacia una víctima puede parecer la opción más rápida, pero puede ser necesario apagar fuego, abrir una puerta o modificar la ruta para evitar una situación más peligrosa. Por ello, el comportamiento de los agentes no se limita a encontrar la distancia más corta entre dos puntos.

Narrativa

Una llamada de emergencia alerta sobre un incendio dentro de un edificio con personas atrapadas. Un equipo de seis bomberos llega al lugar e inicia una operación de búsqueda y rescate mientras el fuego continúa avanzando por la estructura. Algunos puntos del edificio contienen posibles víctimas, pero inicialmente no se sabe si cada marcador representa una persona real o una falsa alarma. Los bomberos deben explorar el edificio, identificar estos puntos de interés, rescatar a las víctimas y controlar el incendio.

Mientras los agentes actúan, el fuego puede generar humo, propagarse hacia espacios cercanos, producir explosiones y dañar paredes o puertas. Esto hace que el escenario cambie constantemente y obliga a los agentes a reconsiderar sus decisiones durante la simulación.

La partida representa un esfuerzo cooperativo donde todos los bomberos ganan o pierden juntos, siguiendo la misma idea central del juego original.

Objetivo del juego

El objetivo principal consiste en rescatar siete víctimas antes de que el incendio provoque una condición de derrota.

La simulación utiliza un tablero bidimensional de 6 filas por 8 columnas, cuya estructura se obtiene desde un archivo de entrada. Inicia con seis agentes bomberos y diferentes elementos del escenario, como fuego, puertas, paredes, posibles víctimas y salidas. El reto establece además el uso de las reglas correspondientes al Family Game Setup.

Condiciones de victoria

La simulación termina con una victoria cuando:

- Se rescatan 7 víctimas.

Condiciones de derrota

La simulación termina con derrota cuando:

- Se pierden 4 víctimas o el edificio colapsó debido al daño estructural.

En la implementación actual se utilizan los 24 marcadores de daño definidos por el reglamento oficial como límite estructural del edificio.

Mecánicas principales

Cada bombero dispone normalmente de 4 Action Points (AP) durante su turno, estos puntos se utilizan para ejecutar distintas acciones. Los AP que no se utilizan pueden conservarse hasta un máximo acumulado de cuatro, siguiendo las reglas utilizadas por la simulación.

Las acciones principales son:

- **Movimiento**

El bombero puede desplazarse hacia una celda adyacente en las direcciones arriba, abajo, izquierda o derecha y no existen movimientos diagonales. Moverse hacia una celda normal cuesta 1 AP, mientras que entrar en una celda con fuego cuesta 2 AP.

- **Apertura de puertas**

Las puertas cerradas bloquean el movimiento normal entre dos celdas, abrir o cerrar una puerta cuesta 1 AP.

- **Extinción**

Un bombero puede combatir fuego o humo en su propia celda o en una celda adyacente.

- Eliminar humo: 1 AP.
- Convertir fuego en humo: 1 AP.
- Eliminar completamente el fuego: 2 AP.

- **Rescate de víctimas**

Cuando un bombero encuentra una víctima, puede cargarla. Mientras transporta una víctima, el costo de movimiento aumenta y no puede desplazarse hacia una celda que contenga fuego y cuando la víctima alcanza una salida del edificio, se considera rescatada.

- **Puntos de interés**

Al entrar a una celda con un POI no identificado, el marcador se revela. Puede contener:

- una víctima;
- una falsa alarma.

Revelar el POI no tiene costo adicional de AP.

- **Romper paredes**

Los bomberos pueden utilizar su equipo para destruir segmentos de pared y crear nuevas rutas. Colocar un marcador de daño sobre una pared cuesta 2 AP, cuando una pared acumula dos marcadores, se considera destruida y puede ser atravesada.

Estas acciones corresponden a las mecánicas principales definidas en las reglas Family del juego original.

Play loop

El ciclo principal de la simulación es:

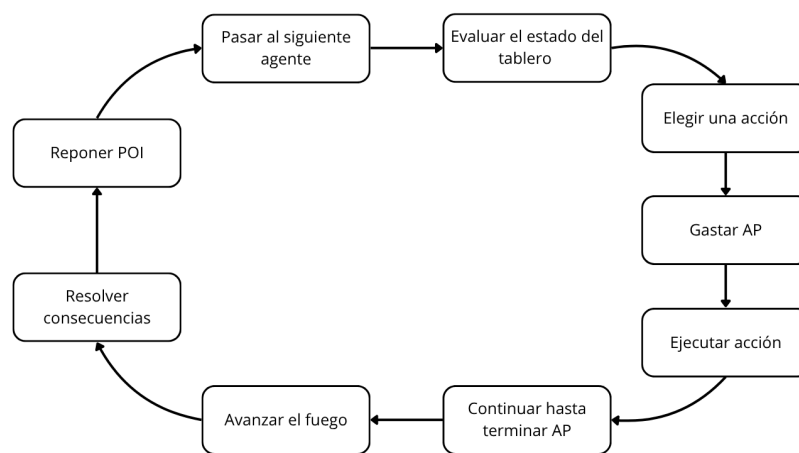


Figura 1. Diagrama del ciclo principal de la simulación o Play Loop.

Este ciclo se repite hasta que ocurre una condición de victoria o derrota.

Durante cada turno, el estado del tablero puede cambiar considerablemente. Una ruta que era segura puede dejar de serlo después del avance del fuego, una puerta puede ser destruida por una explosión o un POI puede desaparecer después de ser identificado. Por esta razón, los agentes de la estrategia mejorada realizan nuevas evaluaciones mientras avanza la partida.

Estrategias de los agentes

Uno de los objetivos centrales del proyecto es comparar dos formas diferentes de tomar decisiones. El reto exige implementar una solución aleatoria y una estrategia desarrollada por el equipo que demuestre una mejora respecto a la primera.

- Estrategia aleatoria

La estrategia aleatoria funciona como línea base para evaluar el proyecto.

El agente primero obtiene todas las acciones que legalmente puede ejecutar de acuerdo con su posición, AP disponibles y estado del tablero. Posteriormente selecciona aleatoriamente una de estas acciones.

Por lo tanto, que una estrategia sea aleatoria no significa que pueda romper las reglas. El comportamiento aleatorio solamente ocurre al momento de elegir entre acciones previamente validadas. Esta estrategia permite medir qué tan eficiente sería un grupo de agentes que conoce las reglas del juego, pero que no cuenta con planeación ni priorización.

- **Estrategia mejorada**

La estrategia mejorada añade un proceso de decisión.

Combina:

- Búsqueda de rutas mediante A*
- Priorización de víctimas y POI
- Evaluación de riesgo producido por fuego
- Apertura de puertas cuando forman parte de una ruta
- Extinción preventiva
- Coordinación entre agentes
- Replaneación ante cambios en el tablero

El orden general de decisión es el siguiente:

1. Si el bombero está cargando a una víctima, prioriza encontrar una salida.
2. Si existe una víctima revelada en su misma posición, la recoge.
3. Revisa si puede eliminar fuego completo o humo peligroso cercano.
4. Evalúa los POI y víctimas disponibles.
5. Selecciona un objetivo utilizando el costo de ruta y diferentes factores de prioridad.
6. Si el siguiente paso contiene fuego, intenta apagarlo antes de avanzar.
7. Si una puerta cerrada bloquea la ruta, la abre.
8. Avanza hacia su objetivo.
9. Si no tiene un objetivo alcanzable, busca contribuir al control del incendio.

Esto permite que el comportamiento sea más dirigido que la simple elección aleatoria.

Sistema investigado: A* y planeación de rutas

Uno de los sistemas principales integrados en la estrategia mejorada es el algoritmo A*.

La versión utilizada toma como referencia el algoritmo trabajado en el material de *Path Planning* del curso y posteriormente se adapta a las características específicas de *Flash Point*. La principal diferencia respecto a una cuadrícula tradicional es que en este proyecto los obstáculos no están representados únicamente por celdas bloqueadas. Las paredes y puertas se encuentran entre dos celdas, por lo que A* debe consultar la topología real del tablero antes de considerar un movimiento como válido.

El algoritmo utiliza:

- g: costo acumulado para llegar hasta una celda
- h: distancia Manhattan hasta el objetivo
- $f = g + h$ prioridad utilizada para explorar las rutas

La distancia Manhattan es apropiada porque los bomberos solamente pueden desplazarse vertical u horizontalmente y no en diagonal.

Adaptación al juego

El costo de una ruta considera varios factores:

- Espacio libre: costo normal de movimiento
- Puerta cerrada: se agrega el costo de abrirla
- Fuego: se agrega una penalización
- Bombero cargando víctima: aumenta el costo de movimiento
- Fuego mientras se transporta una víctima: celda intransitable

Un fragmento relevante del código es:

```
Python
door = board.get_door_between(row, column, *next_cell)

if door is not None and not door.is_open and not door.is_destroyed:
    extra = DOOR_COST
elif not board.can_move_between(row, column, *next_cell):
    continue

if board.has_fire(*next_cell):
    if carrying:
        continue

    cost = 2 + FIRE_PENALTY
else:
    cost = 2 if carrying else 1
```

En la implementación:

Python

FIRE_PENALTY = 2

DOOR_COST = 1

Esto significa que A* no busca necesariamente el recorrido con menor cantidad de celdas, sino la ruta con el menor costo de planeación.

Priorización de objetivos

Además de encontrar rutas, la estrategia necesita decidir a qué objetivo conviene dirigirse.

Cada víctima o POI recibe una puntuación.

De manera general:

$$\begin{aligned} \text{Puntuación} = & \text{costo de la ruta } A^* + \text{penalización por POI desconocido} - \text{urgencia} \\ & + \text{penalización por congestión} - \text{bonificación por mantener el objetivo actual.} \end{aligned}$$

Una víctima revelada tiene mayor prioridad que un POI desconocido porque el POI todavía podría resultar una falsa alarma. También se agrega prioridad cuando existe fuego cerca de la víctima, debido al riesgo de que se pierda antes de que un agente pueda rescatarla.

Finalmente, existe una pequeña bonificación por conservar el objetivo que el agente ya tenía asignado. Esto evita que el bombero cambie constantemente de dirección cuando existen varios objetivos con puntuaciones muy similares.

Coordinación entre agentes

La estrategia mejorada incluye un mecanismo de coordinación para evitar que demasiados bomberos se dirijan innecesariamente hacia el mismo objetivo.

El modelo mantiene una tabla compartida que relaciona:

bombero $\rightarrow$ objetivo asignado

Cuando un bombero evalúa una víctima o POI, revisa cuántos agentes ya se encuentran asignados a esa misma posición y por cada bombero adicional que persigue el mismo objetivo se aplica una penalización

al costo de selección. Esto favorece que los seis agentes se distribuyan entre diferentes objetivos cuando existen alternativas razonables.

El sistema se implementa como una mejor respuesta secuencial, cada agente elige la opción más conveniente tomando en cuenta las decisiones actuales de los demás.

No se calcula un equilibrio global y tampoco existe aprendizaje entre partidas.

Replaneación

La ruta calculada mediante A^* no se conserva permanentemente.

En cada nueva decisión, el agente vuelve a calcular la ruta utilizando el estado actual del tablero.

Esto es necesario porque el entorno cambia constantemente debido a:

- propagación del fuego
- generación de humo
- explosiones
- destrucción de puertas
- daño estructural
- POI identificados
- víctimas rescatadas o perdidas

Por ejemplo, si aparece fuego sobre el camino que un bombero pensaba seguir, la siguiente ejecución de A^* puede encontrar una ruta diferente. El objetivo elegido sí puede mantenerse para evitar cambios innecesarios, pero la ruta hacia ese objetivo se actualiza.

Simulación multiagente con Mesa

El servidor utiliza Mesa 3.5.1 para manejar el modelo basado en agentes.

Para representar las posiciones se utiliza:

```
Python
MultiGrid(width=8, height=6, torus=False)
```

Se eligió MultiGrid porque el reto establece explícitamente que puede haber más de un agente dentro de una misma celda.

El proyecto separa dos responsabilidades:

MultiGrid: posiciones de los bomberos.

Board: paredes, puertas, salidas, fuego, humo, POI y daño estructural.

También se utiliza una vecindad sin diagonales, equivalente a Von Neumann, porque el reglamento especifica que únicamente son adyacentes las celdas ubicadas arriba, abajo, izquierda o derecha.

Arquitectura cliente-servidor

La arquitectura se divide claramente en dos componentes:

- **Servidor — Python / Mesa**

El servidor funciona como el motor de la simulación y es responsable de agentes, reglas, estrategias, AP, fuego y humo, explosiones, daño, POI, víctimas, condiciones de victoria y derrota

- **Cliente — Unity**

Unity funciona como cliente y capa de visualización, no toma decisiones por los bomberos ni replica la simulación. Su responsabilidad es recibir el estado generado por Python y representarlo mediante objetos 3D.

El flujo general es:

Python / Mesa → HTTP → JSON → SimulationClient → BoardManager → Unity 3D

API del servidor

El servidor expone los siguientes endpoints:

Endpoint	Función
GET /board	Obtener la geometría y configuración fija del tablero.
GET /state	Obtener el estado actual de la simulación.
POST /step	Ejecutar un turno y devolver el nuevo estado.
POST /reset	Reiniciar la simulación con una semilla y estrategia determinadas.

Tabla 2. Endpoints del servidor y sus funciones.

Unity utiliza `SimulationClient.cs` para realizar estas peticiones.

Al iniciar, puede ejecutar un `POST /reset` para seleccionar la estrategia y la semilla. Posteriormente solicita un `POST /step` cada cierto intervalo y actualiza la escena con la respuesta.

Visualización e interacción

El reto solicita una visualización en Unity mediante una vista superior que utiliza modelos tridimensionales y permite navegar por el escenario. El usuario no controla directamente a los seis bomberos, su función principal es observar la simulación mientras los agentes actúan de forma autónoma.

La versión actual utiliza prefabs para representar:

- Celdas, paredes, puertas, salidas, bomberos, fuego, POI, víctimas, falsas alarmas

Controles de cámara

Control	Acción
W, A, S, D	Desplazar la cámara
Flechas	Desplazar la cámara
Rueda del mouse	Acercar o alejar
F	Recentrar el tablero

Tabla 3. Controles de cámara

La cámara no modifica el estado de la simulación y solamente funciona como una capa visual.

Referencias visuales y mejoras



Bomberos. Se ajustó la escala de los modelos para que tuvieran mayor presencia en el tablero, ya que antes se perdían entre las celdas y se pulió para que el recorrido entre casillas se sintiera fluido en vez de un salto brusco de celda en celda.

Ambientación. El escenario dejó de verse tan vacío: se incorporó mobiliario (mesas, sillas, televisores, entre otros elementos) para que cada habitación se leyera de inmediato como un espacio habitado y no como un tablero vacío.

Interfaz. Se integró la primera versión del HUD, con los indicadores necesarios de la partida

Materiales. Se utilizaron texturas de piso y pared pensadas específicamente para transmitir la sensación de estar dentro de una casa real, y así hacer una especie de inmersión en el juego .



Efectos del fuego. El fuego se construyó de la siguiente manera: llamas animadas y chispas que se desprenden hacia arriba, esto con el objetivo de que realmente parezcan llamas producidas por un incendio.

Efectos de los POI. Los puntos de interés sin revelar laten con un ligero cambio de escala, a ritmo desfasado entre sí, para que los usuarios puedan identificarlo de manera más sencilla

Seguimiento de los bomberos. Cada uno de los bomberos tiene una diminuta estela de luz que lo persigue por donde sea que este camine, este solo es un detalle para poder darnos cuenta de que dirección es que viene el bombero.

Créditos de assets

Los assets utilizados en este proyecto fueron creados con apoyo de Claude (Anthropic) como herramienta de inteligencia artificial generativa. Estos recursos fueron posteriormente revisados, adaptados e integrados al proyecto de acuerdo con las necesidades del desarrollo.

Comparación experimental

Para comprobar que la estrategia mejorada realmente supera a la solución aleatoria se realizaron pruebas utilizando 100 semillas pareadas, de 0 a 99.

Las tres estrategias recibieron:

- El mismo tablero
- El mismo mazo de POI
- Misma secuencia de tiradas aleatorias del fuego

Para evitar que la estrategia aleatoria modificara accidentalmente los eventos del entorno, el proyecto separa tres generadores aleatorios:

```
Python
fire_random → avance del fuego
poi_random → POI y reposición
strategy_random → decisiones aleatorias
```

Esto permite hacer una comparación justa entre estrategias.

Resultados promedio

Métrica	Aleatoria	Mejorada sin coordinación	Mejorada
Victorias	0%	38%	44%
Colapsos	100	51	45
Víctimas rescatadas	0.05	5.05	5.20
Víctimas perdidas	0.52	1.74	1.70
Daño estructural	24.00	19.49	19.08
Turnos	20.76	40.20	39.96
Derribos	20.86	7.38	7.99
Replaneaciones	0.00	1.28	2.52
Fuegos apagados	11.16	16.22	16.42
Tiempo por partida	4.56 ms	55.75 ms	33.8 ms

Tabla 4. Resultados promedio para comparación entre estrategias.

La diferencia entre la estrategia mejorada y la aleatoria es clara: la aleatoria no obtuvo ninguna victoria en las 100 semillas utilizadas, mientras que la mejorada logró 44. También aumentó el promedio de víctimas rescatadas de 0.05 a 5.20 y redujo el daño estructural promedio de 24.00 a 19.08.

La estrategia mejorada requiere más procesamiento debido al uso de A*, priorización y coordinación, pero sigue ejecutándose en tiempos de milisegundos por partida.

Aporte de la coordinación

Para medir específicamente el efecto de la coordinación se añadió una tercera variante:

- Estrategia mejorada sin coordinación.

Esta conserva A*, priorización y control del incendio, pero ignora las asignaciones de los demás bomberos.

Sus resultados fueron:

- Mejorada sin coordinación: 38% victorias.
- Mejorada con coordinación: 44% victorias.

La diferencia favorece a la versión coordinada, pero con las 100 semillas oficiales no es estadísticamente significativa. Por esta razón, el proyecto no presenta el 38% frente al 44% como prueba definitiva de que la coordinación produce mejores resultados.

Pruebas y reproducibilidad

El proyecto cuenta con 150 pruebas automatizadas, todas ejecutadas correctamente en el entorno local utilizado para la validación final.

```
Python
Ran 150 tests
OK
```

Después de integrar MultiGrid, las pruebas se volvieron a ejecutar sin modificar las métricas obtenidas anteriormente.

También se verificó la reproducibilidad de los experimentos entre dos equipos diferentes.

Se compararon 5,700 celdas lógicas de resultados, obteniendo 0 diferencias. La única métrica que cambia entre computadoras es el tiempo de ejecución, ya que depende del hardware y del entorno, esto permite afirmar que los resultados lógicos de la simulación son reproducibles.

Alcance del proyecto

El alcance se limita deliberadamente a las reglas correspondientes al Family Game Setup, tal como especifica el reto. Por este motivo no se implementaron características pertenecientes a las reglas Experienced, entre ellas:

- ambulancia
- camión de bomberos
- especialistas
- Hazmat
- Hot Spots
- Flare Ups
- acciones especiales de vehículos

Estas mecánicas existen en el juego original, pero no eran necesarias para cubrir el alcance solicitado, también se decidió priorizar el desarrollo del motor de reglas, las estrategias, las pruebas, la reproducibilidad y la comunicación cliente-servidor antes que el nivel final de acabado visual.

Aunque la versión final ya incorpora HUD, ambientación, materiales, efectos de fuego y elementos visuales para los POI, todavía existe margen para aumentar el nivel de acabado gráfico. Entre las mejoras futuras se encuentran modelos 3D con mayor detalle, una diferenciación visual todavía más marcada entre fuego y humo, refinamiento de iluminación, animaciones adicionales y una mayor variedad de elementos decorativos.

Innovación

El principal aporte propio del proyecto se encuentra en la combinación de distintas técnicas dentro de una misma estrategia multiagente.

La solución mejorada integra:

- búsqueda A* adaptada a paredes y puertas colocadas entre celdas
- costo adicional por riesgo de incendio
- priorización dinámica de víctimas y POI
- detección de urgencia

- permanencia de objetivos para evitar oscilaciones
- coordinación mediante asignaciones compartidas
- replaneación automática ante cambios del escenario

La estrategia no modifica las reglas de Flash Point, la innovación se encuentra en cómo los agentes toman decisiones dentro de esas reglas. Esto permite comparar experimentalmente distintos niveles de inteligencia manteniendo exactamente el mismo entorno de simulación.

Uso de inteligencia artificial generativa

Durante el desarrollo se utilizó Claude, de Anthropic y ChatGPT de OpenIA como herramienta de apoyo para distintas tareas de programación, adaptación de algoritmos, pruebas y documentación en tema de apoyo con organización y redacción del presente documento. El proyecto conserva documentación sobre el origen del código y archivos de referencia utilizados previamente por integrantes del equipo.

La IA se utilizó como herramienta de apoyo y no como sustitución de la validación del proyecto. Las pruebas automatizadas y los experimentos incluidos en los resultados fueron ejecutados sobre el código real del proyecto.

Después de finalizar el backend se verificaron:

- 150 de 150 pruebas automatizadas
- experimentos de 100 semillas
- integración Python–Unity
- comportamiento de la cámara
- reproducibilidad de resultados

Reflexión de equipo

Durante el desarrollo de este proyecto aprendimos que una simulación multiagente implica mucho más que hacer que varios agentes se muevan de manera independiente. Como equipo, entendimos que lo más importante era lograr que cada decisión respetara las reglas del juego y que, al mismo tiempo, los agentes pudieran reaccionar ante un escenario que cambia constantemente. Una de las partes que más trabajo nos costó fue integrar A* con elementos como puertas, fuego, víctimas y diferentes costos de movimiento.

También se nos hizo interesante la diferencia entre tener agentes que simplemente siguen las reglas y tener agentes que realmente toman decisiones más eficientes. La estrategia aleatoria puede ejecutar acciones válidas, pero sus resultados fueron mucho menores que los de la estrategia mejorada, esto nos ayudó a

comprender mejor la importancia de la planeación, la priorización de objetivos y la coordinación dentro de un sistema multiagente.

Criterio	Evidencia
Solución Aleatoria	El bombero aleatorio solo puede elegir entre las jugadas que en ese momento son realmente válidas, así que nunca hace trampa ni rompe reglas, simplemente no juega con "inteligencia". Interactúa con todo lo que hay en el tablero: se mueve, abre puertas, apaga fuego, carga víctimas. Lo probamos muchísimas veces y nunca se comportó fuera de las reglas.
Estrategia mejorada	Corrimos las dos estrategias 100 veces cada una, siempre bajo exactamente las mismas condiciones, para que la comparación si fuera justa. La aleatoria nunca ganó, aunque eventualmente lo hubiera hecho, la mejorada ganó casi la mitad de las veces, rescató muchas más víctimas y causó menos daño al edificio. Además confirmamos que los números salían igual en dos computadoras distintas.
Rendimiento Obtenido	La simulación corre bien y los bomberos si se ven trabajando en equipo, evitan amontonarse todos sobre la misma víctima, además de repartirse el edificio entre ellos.
Cliente-Servidor	Unity y la parte de Python se comunican sin problemas, Unity pide el turno y Python responde con lo que pasó. Además, armamos una revisión automática que nos avisa de inmediato si algún dato deja de coincidir entre los dos lados, esto con el objetivo de reducir el margen de error
Visualización	Algunos modelos hechos por nosotros, fuego que se ve vivo, y un HUD con los datos importantes de la partida, además de la cámara que se puede mover y

	alejar para ver todo el tablero, los puntos de interés laten para llamar la atención, y cada bombero se mueve de forma suave y tiene su propio color para poder seguirlo con la vista.
Narrativa	La historia la planteamos como una llamada de emergencia real, mantiene la incertidumbre de no saber si un punto de interés es una víctima o una falsa alarma, y el final depende del desempeño de todo el equipo, igual que en el juego de mesa original. La idea es crear un experiencia inmersiva en el juego, hacer sentir al espectador que él es un bombero a punto de entrar a salvar víctimas.

Fuentes y referencias

- García Galván, V. A., Gómez Ayala, C. A., & Martínez Barragán, L. F. (2026). *Flashpoint-Fire-Rescue* [Código fuente]. GitHub. <https://github.com/vic-gar/Flashpoint-Fire-Rescue>
- Lanzing, K. (s. f.). *Flash Point: Fire Rescue* [Reglamento del juego]. Indie Boards & Cards.
- Reto AD 2026. (2026). [Documento de especificación del proyecto]. Instituto Tecnológico y de Estudios Superiores de Monterrey.
- Rúbrica para el Proyecto de Simulación de Búsqueda y Rescate. (2026). [Rúbrica de evaluación]. Instituto Tecnológico y de Estudios Superiores de Monterrey.